

Production-Level Fintech Expense Tracker System

Project Vision

A fintech platform that allows users to:

- Login securely
- Connect simulated bank accounts
- Track transactions
- Analyze spending behavior
- View AI-driven analytics
- Categorize expenses automatically
- Monitor financial health

This version focuses on:

- Full Frontend
- Authentication System
- Analytics Dashboard
- Transaction Categorizer
- Simulated Bank Sync

This is the correct production-style MVP before integrating real banking APIs.

1. Recommended Tech Stack

Layer	Technology
Frontend	React + TypeScript
Styling	Tailwind CSS
State Management	Redux Toolkit
Charts	Recharts
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	JWT + Refresh Tokens
AI Categorization	Python NLP Engine

Layer	Technology
Deployment	Docker
API Testing	Postman
Version Control	Git + GitHub

2. Complete System Architecture

```
Frontend (React)
  ↓
API Gateway (FastAPI)
  ↓
Authentication Service
  ↓
Transaction Service
  ↓
Categorization Engine
  ↓
Analytics Engine
  ↓
PostgreSQL Database
```

3. Production-Level Frontend Structure

```
frontend/
|
|─ public/
|─ src/
|   |─ assets/
|   |─ components/
|   |   |─ dashboard/
|   |   |─ auth/
|   |   |─ charts/
|   |   |─ transactions/
|   |   |─ layouts/
|   |   └─ ui/
|   |─ pages/
|   └─ Login.tsx
```

```
|
|
|   ├── Register.tsx
|   ├── Dashboard.tsx
|   ├── Analytics.tsx
|   ├── Transactions.tsx
|   └── Settings.tsx
|
|   ├── redux/
|   |   ├── store.ts
|   |   ├── authSlice.ts
|   |   ├── transactionSlice.ts
|   |   └── analyticsSlice.ts
|   |
|   ├── services/
|   |   ├── api.ts
|   |   ├── authService.ts
|   |   ├── transactionService.ts
|   |   └── analyticsService.ts
|   |
|   ├── hooks/
|   ├── routes/
|   ├── utils/
|   ├── App.tsx
|   └── main.tsx
```

4. Backend Structure

```
backend/
|
|   ├── app/
|   |   ├── api/
|   |   |   ├── auth.py
|   |   |   ├── transactions.py
|   |   |   ├── analytics.py
|   |   |   └── bank_sync.py
|   |   |
|   |   ├── models/
|   |   |   ├── user.py
|   |   |   ├── transaction.py
|   |   |   └── bank_account.py
|   |   |
|   |   ├── services/
|   |   |   ├── auth_service.py
|   |   |   └── analytics_service.py
```

```
| | | | | categorizer.py
| | | | | bank_sync_service.py
| | | | |
| | | | | database/
| | | | | | | connection.py
| | | | | | | base.py
| | | | |
| | | | | middleware/
| | | | | schemas/
| | | | | utils/
| | | | | main.py
| | | | |
| | | | | requirements.txt
| | | | | Dockerfile
| | | | | docker-compose.yml
```

5. Frontend Features

A. Authentication UI

Pages

Login Page

Features:

- Email/mobile login
- Password validation
- JWT token handling
- Remember device
- Forgot password

Register Page

Features:

- Full name
 - Mobile number
 - Email
 - Password
 - Confirm password
-

B. Dashboard UI

Main Dashboard Cards

- Total Balance
 - Monthly Spending
 - Monthly Income
 - Savings Percentage
 - Recent Transactions
 - Budget Remaining
-

C. Charts Section

Charts

Pie Chart

Expense category distribution

Line Chart

Monthly spending trend

Bar Chart

Top merchants

Heatmap

Daily transaction activity

D. Transactions Module

Features

- Search transactions
- Filter by category
- Filter by date
- Sort by amount
- Transaction details modal
- Download statement

6. Authentication System

JWT Authentication Flow

```
User Login
  ↓
Backend verifies credentials
  ↓
Access Token Generated
  ↓
Refresh Token Generated
  ↓
Stored securely
```

Password Security

Use:

- bcrypt hashing
- token expiration
- refresh token rotation
- secure cookies

Recommended Backend Auth Libraries

```
fastapi
python-jose
passlib
bcrypt
```

Example User Model

```
class User(Base):
    __tablename__ = "users"
```

```
id = Column(Integer, primary_key=True)
name = Column(String)
email = Column(String, unique=True)
phone = Column(String)
hashed_password = Column(String)
```

7. Simulated Bank Sync System

Why Simulated Sync?

Before integrating real banking APIs, simulate:

- bank accounts
- transaction feeds
- balances
- transfers

This allows:

- frontend testing
- analytics testing
- AI categorization
- scalability testing

Simulated Bank Architecture

```
Mock Bank API
  ↓
Sync Service
  ↓
Transaction Processor
  ↓
Database
```

Mock Bank Data Example

```
{
  "account_number": "XXXX1234",
  "transactions": [
    {
      "amount": 450,
      "merchant": "Swiggy",
      "type": "debit",
      "date": "2026-05-07"
    }
  ]
}
```

Simulated Sync API

Endpoint

```
POST /sync-bank
```

Example FastAPI Route

```
@router.post("/sync-bank")
def sync_bank(user_id: int):
    transactions = fetch_mock_transactions()

    for txn in transactions:
        save_transaction(txn)

    return {"message": "Bank sync completed"}
```

8. Transaction Categorization Engine

Objective

Automatically classify:

- Food
 - Travel
 - Shopping
 - Recharge
 - Utilities
 - Entertainment
 - Healthcare
 - Investments
-

Categorization Strategy

Stage 1 — Rule-Based Engine

```
if "swiggy" in merchant:  
    category = "Food"
```

Stage 2 — ML Categorizer

Use:

- Scikit-learn
 - NLP
 - Merchant embeddings
-

Categorization Pipeline

```
Transaction  
↓  
Merchant Extraction  
↓
```

```
Rule Engine
  ↓
ML Model
  ↓
Final Category
```

9. Analytics Engine

Features

Spending Analytics

- monthly trends
 - category insights
 - overspending alerts
 - savings ratio
-

Financial Health Score

Example calculation:

```
Savings > 30% → Excellent
Savings 15-30% → Good
Savings < 15% → Poor
```

Monthly Analytics Response

```
{
  "monthly_spending": 25000,
  "top_category": "Food",
  "savings_percentage": 22
}
```

10. Database Schema

USERS TABLE

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  password TEXT  
);
```

TRANSACTIONS TABLE

```
CREATE TABLE transactions (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER,  
  amount NUMERIC,  
  merchant VARCHAR(255),  
  category VARCHAR(100),  
  transaction_type VARCHAR(50),  
  created_at TIMESTAMP  
);
```

BANK ACCOUNTS TABLE

```
CREATE TABLE bank_accounts (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER,  
  bank_name VARCHAR(100),  
  account_number VARCHAR(50)  
);
```

11. Production-Level API Endpoints

AUTH APIs

```
POST /register  
POST /login  
POST /refresh-token  
POST /logout
```

TRANSACTION APIs

```
GET /transactions  
GET /transactions/{id}  
POST /sync-bank
```

ANALYTICS APIs

```
GET /analytics/monthly  
GET /analytics/categories  
GET /analytics/insights
```

12. Dashboard UI Design

Sidebar

- Dashboard
 - Transactions
 - Analytics
 - Budgets
 - Settings
-

Main Content

- Balance summary
 - Charts
 - Transaction list
 - AI insights
 - Expense categories
-

13. Security Requirements

Mandatory Security Features

Backend

- JWT validation
 - bcrypt password hashing
 - rate limiting
 - API throttling
 - CORS protection
-

Frontend

- protected routes
 - token expiration handling
 - secure local storage
 - session management
-

Database

- encrypted backups
 - role-based access
 - transaction logs
-

14. Deployment Architecture

Docker Setup

```
Frontend Container  
Backend Container  
PostgreSQL Container  
Redis Container
```

Production Deployment

Use:

- AWS EC2
- NGINX
- Docker Compose
- GitHub Actions CI/CD

15. Recommended Development Sequence

STEP 1

Build backend authentication

STEP 2

Create frontend authentication pages

STEP 3

Develop dashboard layout

STEP 4

Build transaction APIs

STEP 5

Implement simulated bank sync

STEP 6

Develop analytics engine

STEP 7

Build categorization system

STEP 8

Deploy with Docker

16. Recommended Frontend Libraries

Purpose	Library
Charts	Recharts
Routing	React Router
Forms	React Hook Form
Validation	Zod
HTTP	Axios
State Management	Redux Toolkit
Notifications	React Hot Toast

Purpose	Library
Icons	Lucide React

17. Production-Level Improvements

Future Features

AI Features

- AI savings advisor
- spending prediction
- fraud detection
- smart budgeting

Fintech Features

- Account Aggregator integration
- real bank sync
- UPI analytics
- subscription tracking

Enterprise Features

- audit logs
- admin dashboard
- multi-device login
- notification system

18. Realistic Timeline

Phase	Duration
Frontend	2 Weeks
Authentication	1 Week
Transactions Module	1 Week

Phase	Duration
Dashboard Analytics	2 Weeks
Categorizer	1 Week
Simulated Bank Sync	1 Week
Testing + Deployment	1 Week

Estimated Total:

8-10 Weeks

19. Final Recommendation

Do NOT immediately attempt:

- direct UPI APIs
- NPCI integrations
- banking partnerships
- real-time financial APIs

Instead:

1. Build a production-quality MVP
2. Simulate bank systems
3. Build analytics intelligence
4. Improve categorization accuracy
5. Deploy publicly
6. Then approach fintech integrations

This is the realistic startup-level development path.